



EG1113 - C Programming for Engineering Lab

Report #5

Arrays and PWM

Date of the experiment: September 25, 2024

Due date of report: September 27, 2021

Name: Nicholas Galus

1. Core concepts learned

- Setup VS file in VS code
- Going through a multi-dimensional array with a for loop

2. List of equipment and software used

- Raspberry PI 400
- Lenovo Laptop
- Caja File Network Share
- VS Code

3. Relevant web links and other documents:

- Utilized [PIGPIO Library](#) to adjust the PWM for the LEDs on the protoboard

4. Exercise Building the Starter Code

4.1 Summary

- Setting up the VS file in order to run the program
- Understanding the starter code, specifically the `set_PWM_dutycycle`, and running it

`set_PWM_dutycycle(user_gpio, dutycycle)`

Starts (non-zero dutycycle) or stops (0) PWM pulses on the GPIO.

Parameters

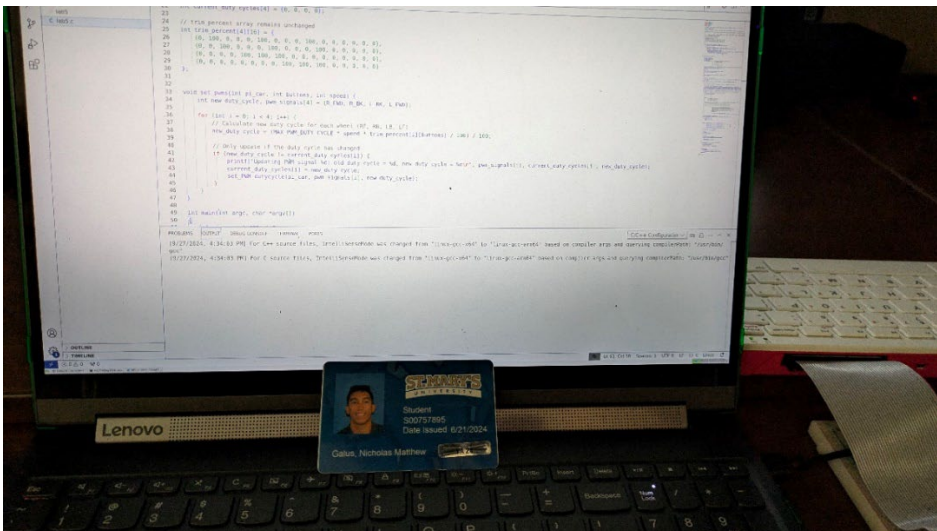
`user_gpio`:= 0-31.
`dutycycle`:= 0-range (range defaults to 255).

The `set_PWM_range` function can change the default range of 255.

Example

```
pi.set_PWM_dutycycle(4, 0) # PWM off
pi.set_PWM_dutycycle(4, 64) # PWM 1/4 on
pi.set_PWM_dutycycle(4, 128) # PWM 1/2 on
pi.set_PWM_dutycycle(4, 192) # PWM 3/4 on
pi.set_PWM_dutycycle(4, 255) # PWM full on
```

- Lab Verification:



4.2 Procedures

1. Download and unzip the starter code to your pi
2. Open program in VS Code and update the program to include the VS file from a previous lab
3. Run the program and understand the code

5. Exercise Raspberry Pi Set Up (Repeat for each exercise)

5.1 Summary

- Modify the program to only call the set_PWM_dutycycle only once
- Lab verification:

5.2 Procedures

1. Modify the function set_pwm to only call the set_PWM_dutycycle only once utilizing for loops and a multi dimensional array called trim_percent

```

25  int trim_percent[4][16] = {
26      {0, 100, 0, 0, 0, 100, 0, 0, 0, 100, 0, 0, 0, 0, 0, 0},
27      {0, 0, 100, 0, 0, 0, 100, 0, 0, 0, 100, 0, 0, 0, 0, 0},
28      {0, 0, 0, 0, 100, 100, 100, 0, 0, 0, 0, 0, 0, 0, 0, 0},
29      {0, 0, 0, 0, 0, 0, 0, 0, 100, 100, 100, 0, 0, 0, 0, 0}
30  };
31
32
33  void set_pwm(int pi_car, int buttons, int speed) {
34      int new_duty_cycle, pwm_signals[4] = {R_FWD, R_BK, L_BK, L_FWD};
35
36      for (int i = 0; i < 4; i++) {
37          // Calculate new duty cycle for each wheel (RF, RB, LB, LF)
38          new_duty_cycle = (MAX_PWM_DUTY_CYCLE * speed * trim_percent[i][buttons] / 100) / 100;
39
40          // Only update if the duty cycle has changed
41          if (new_duty_cycle != current_duty_cycles[i]) {
42
43              current_duty_cycles[i] = new_duty_cycle;
44              set_PWM_dutycycle(pi_car, pwm_signals[i], new_duty_cycle);
45          }
46      }
47  }

```

2. Add a printf statement that verifies that the set_PWM_dutycycle() only happens when the duty cycle needs to change

```

40      // Only update if the duty cycle has changed
41      if (new_duty_cycle != current_duty_cycles[i]) {
42          printf("Updating PWM signal %d: old duty cycle = %d, new duty cycle = %d\n", pwm_signals[i], current_duty_cycles[i], new_duty_cycle);
43          current_duty_cycles[i] = new_duty_cycle;
44          set_PWM_dutycycle(pi_car, pwm_signals[i], new_duty_cycle);
45      }

```

3. Program output:

```

connecting local buttons to 192.168.7.2
using lights on local button board to simulate car

```

Checking for Wireless connection information

```

wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
        inet 10.102.22.222 netmask 255.255.0.0 broadcast 10.102.255.255

```

Successfully connected to the buttons and a car (or car simulator)...here we go

```

Updating PWM signal 19: old duty cycle = 0, new duty cycle = 127
Updating PWM signal 19: old duty cycle = 127, new duty cycle = 0
Updating PWM signal 16: old duty cycle = 0, new duty cycle = 127
Updating PWM signal 16: old duty cycle = 127, new duty cycle = 0
Updating PWM signal 19: old duty cycle = 0, new duty cycle = 127
Updating PWM signal 19: old duty cycle = 127, new duty cycle = 51
Updating PWM signal 19: old duty cycle = 51, new duty cycle = 76
Updating PWM signal 19: old duty cycle = 76, new duty cycle = 102

```

6. Daydream

- Smoother transition between the PWM transition when utilizing the number keys, not jumping from 30% to 80%, but gradually shifting to 80%
- Utilize RGB LEDs to fluctuate between colors when pressing the number keys
- Press one of the number keys to a predetermined value and the light would stay on, like cruise control
- By utilizing a trim, you can fine tune the motors to be able to precisely drive a car to prevent any discrepancies, and for more precision when driving, like turning exactly 30% to the right and not some random value

7. Problems experienced and areas of concern

- No noteworthy problems or concerns to list

8. Course goals

Rank the level (0=none to 5=a large amount) that this lab covered the each of the course goals by updating the appropriate box in the table below.

Course Goals	Laboratory												
	01	02	03	04	05	06	07	08	09	10	11	12	13
To compile, link, debug and run C programs	4	5	5	5	5								
To use the basic C program structure; variables, constants, and operators	4	4	4	4	4								
To use the repetition constructs such as looping with for, while, and do-while statements	2	3	4	4	4								
To use the selection structures such as if, if/else, switch, conditional expression statement	0	0	3	3	3								
To create program modules using functions (passing data to and returning values from functions)	0	0	0	2	4								
To use arrays and pointers	0	0	0	0	4								
To work with strings and string manipulating functions	2	4	4	1	2								
To perform file, I/O operations	3	4	4	5	5								
To develop structured, modular, and top-down design of software	0	0	0	1	1								